

개발완료보고서

IcePredict

AI 예측과 RTOS 반응의 이중 안전망 블랙아이스 대응 시스템

AI가 노면을 미리 보고, 못 보더라도 RTOS가 차량 거동으로 확실히 잡는다.

제24회 임베디드SW경진대회 · 자동차/모빌리티 부문 (현대자동차)

팀 **ARTS** · 이지성 · 남윤상 · 김진찬

STM32N6570-DK · Neural-ART NPU · ThreadX · Raspberry Pi 5 + AI HAT+ 2 · CARLA HIL

github.com/gkgk0119gmail-arch/2026ESWContest_mobility_ARTS



AI가 노면을 미리 보고, 못 보더라도 **RTOS**가 차량 거동으로 확실히 잡는다.

문제

블랙아이스는 공개 데이터에 라벨이 없다. 투명한 얼음은 아스팔트와 시각적으로 구분되지 않는다. 카메라 한 겹으로는 반드시 놓치는 경우가 남는다.

해법

예측과 반응을 한 보드에 두 겹으로 올렸다. 1차는 NPU 비전이 빙판에 닿기 전에 세우고, 1차가 놓치면 2차가 IMU 거동으로 미끄러짐을 확정해 받는다.

증명

실물 STM32N6 보드가 실제 펌웨어를 돌리고 CARLA 가 센서·물리를 제공하는 **HIL** 로 주행 108회, 실사진 69,358장을 보드에 직접 넣어 측정했다.

96.1%

실사진 블랙아이스 6,340장 정답률
경보율 96.6 %

0.3%

마른 노면 19,018장 오경보율
젖은 노면 0.8 %

22.6 μ s

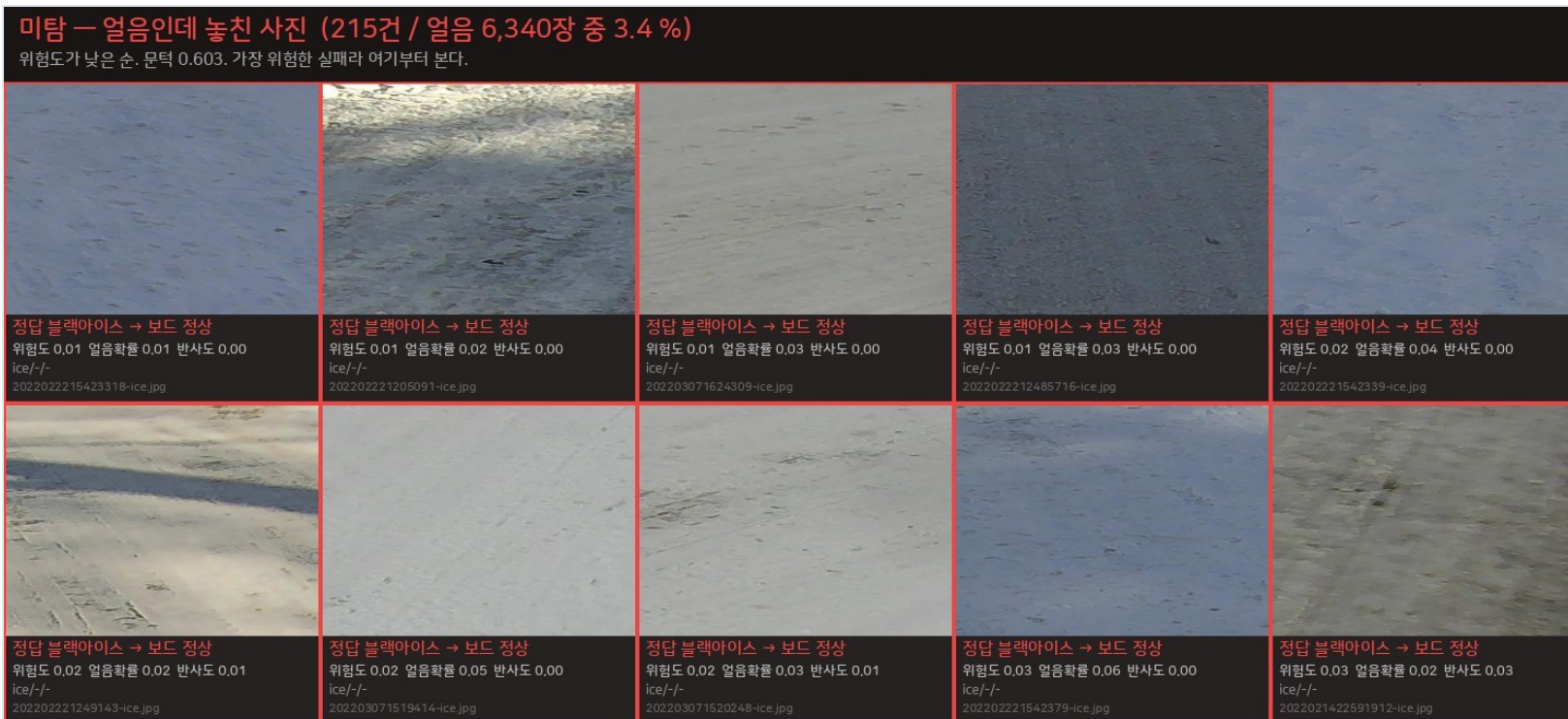
2차 방어 최악 응답
리눅스는 5,790 μ s

9/9

방어를 모두 끈 기준선
전부 제어 상실



같은 빙판 · 같은 속도 · 같은 주변 차량. 왼쪽은 방어 없음 — 제어를 잃고 25.4 km/h 로 차로를 가로질러 미끄러진다. 오른쪽은 STM32N6 RTOS 2차 방어 — 차선 안에서 정지했다



우리 보드가 실제로 놓친 실사진 — 전부 정답이 블랙아이스인데 '정상'으로 판정했다. 위험도 0.01~0.04 라 운영 문턱(0.603)은 물론 어떤 문턱으로도 잡히지 않는다

라벨이 없다

공개 데이터셋에 '블랙아이스' 라벨이 없다. RSCD 의 ice 57,262장도 다져진 눈·서리에 가까운 '얼음 노면'일 뿐, 투명한 블랙아이스를 따로 구분하지 않는다.

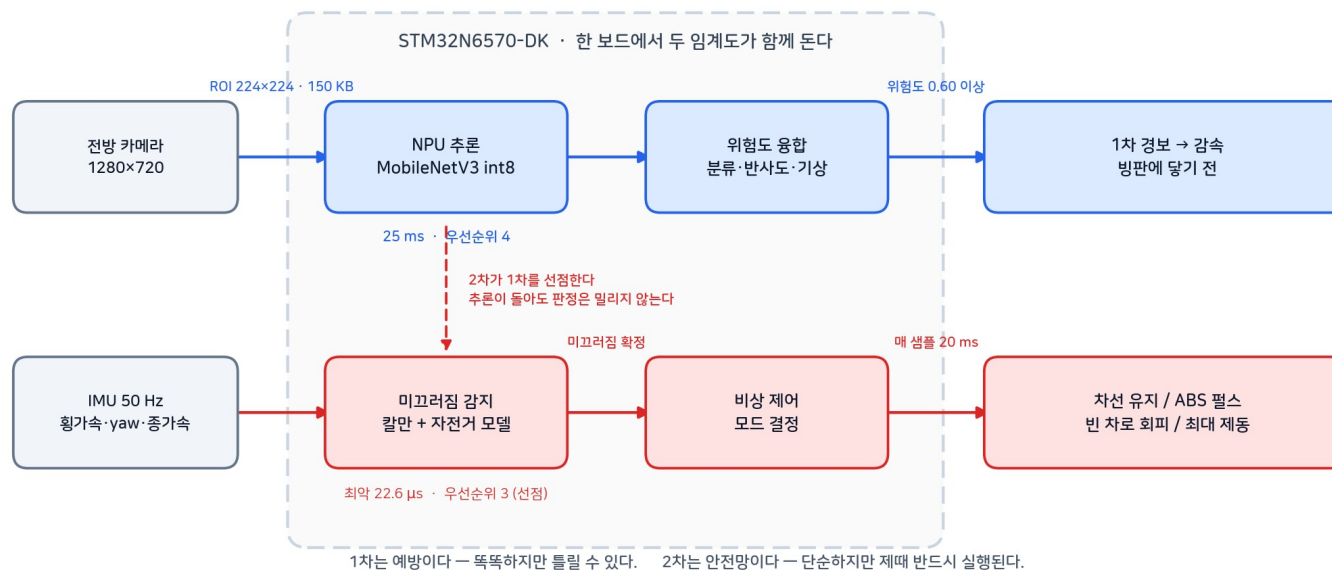
원래 어려운 문제다

투명·검은 얼음이 아스팔트와 시각적으로 구분되지 않는다는 것은 업계 통설이다. 우리 모델도 마찬가지로, 실사진 얼음 6,340장 중 **215장(3.4 %)**이 문턱 아래에 남는다.

그래서 내린 결론

1차는 블랙아이스를 직접 맞히려 하지 않는다. '결빙 위험 노면 확률 + 반사도 이상 + 기상 맥락'으로 위험도를 올린다. 그리고 그 전략이 실패할 때를 위해 2차가 있다.

약점을 숨기지 않고, 이중 안전망 구조의 '존재 이유'를 데이터로 뒷받침한다 — 개발계획서에 적었던 방침을 그대로 지켰다.



	1차 방어 · 예측	2차 방어 · 반응
센서	전방 카메라	IMU (횡가속 · yaw rate · 종가속)
판단	노면 4분류 + 반사도 + 기상 맥락 → 위험도	칼만 필터 + 자전거 모델 잔차 → 미끄러짐 확정
시점	빙판에 닿기 전	미끄러지기 시작한 뒤
성격	똑똑하지만 틀릴 수 있다	단순하지만 제때 반드시 실행된다
실행	STM32N6 NPU · 25 ms	STM32N6 + ThreadX · 최악 22.6 μ s

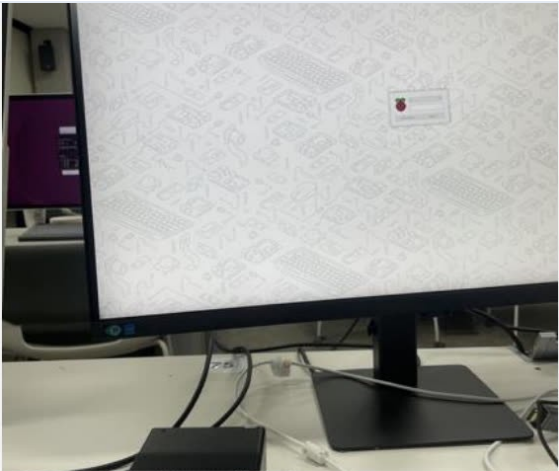
ThreadX 우선순위는 IMU 융합 스레드가 3, NPU 프레임 스레드가 4다. 25 ms짜리 추론이 돌고 있어도 IMU 판정이 선점한다 — 혼합 임계도(mixed-criticality) AI ECU 구조.



Pi 5 16 GB + AI HAT+ 2 (Hailo-10H)
KKS case · active cooling



RealSense D435i (RGB·depth·IMU)
NVMe 1 TB external



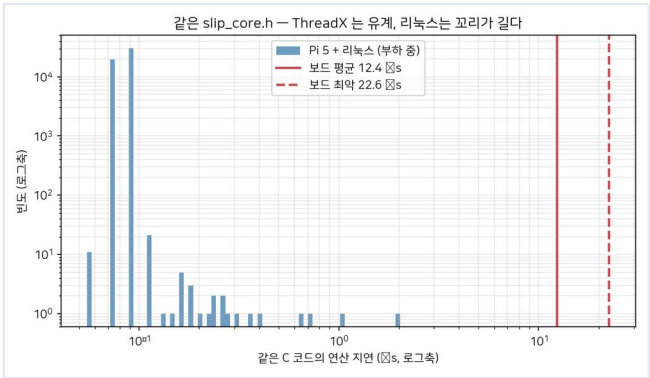
Raspberry Pi OS boot · boot confirmation



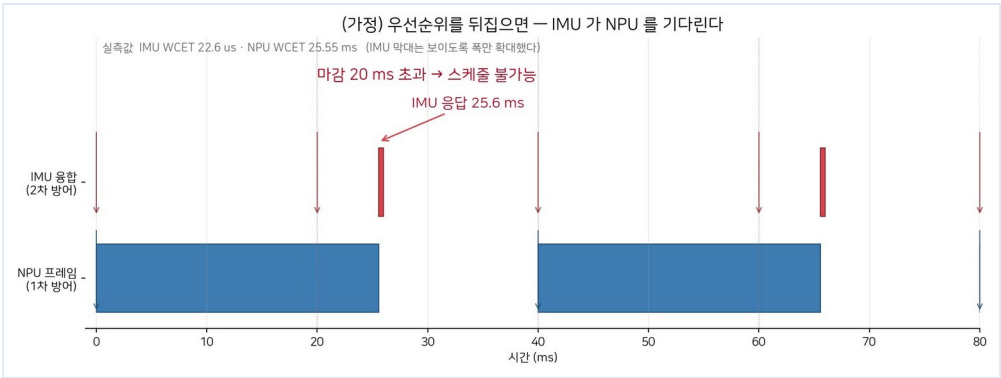
HIL driving — board is judged and CARLA provides physics

구성	역할
STM32N6570-DK	1st NPU inference (Neural-ART @ 1 GHz) + 2nd ThreadX real-time judgment/control. Two task schedulers share one board
Raspberry Pi 5 16 GB + AI HAT+ 2	Host. Weather map generation, ZMQ communication bridge, intersection verification NPU (mandatory board)
RealSense D435i	RGB · depth · IMU. Real-time noise characteristics verification
RTX 5090 workstation	CARLA 0.9.16 — HIL environment physics/sensor provision

항목	개발계획서(6월)	실제 구현	바꾼 이유
2차 방어 실행 환경	Pi 5 + 리눅스	STM32N6 + ThreadX RTOS	리눅스 최악 깨어남 지연이 5,790 μs 로 측정됐다. 제어 주기 20 ms의 29 %를 한 번의 지터가 먹는다. 평균이 아니라 꼬리가 안전 기능의 기준이다
1차 NPU	Hailo-10H 단독	Neural-ART(N6) 주 · Hailo 교차검증	Hailo Dataflow Compiler 를 확보하지 못해 int8 변환 경로를 열지 못했다. 1·2차를 한 보드에 올리는 편이 선점 구조를 증명하기에도 맞다
검증 방법	1/5 차량 저마찰 노면 실측	HIL (실물 보드 + CARLA) + 실사진 69,358장	실물 빙판을 재현 가능하게 만들기 어렵다. 대신 펌웨어는 실물 보드에서 그대로 돌리고 , 비전 성능은 시물 화면이 아닌 실제 도로 사진으로 측정했다
노면 클래스	정상 / 젖음 / 결빙 위험 / 포트홀	동일 (유지)	RSCD 27클래스 → 4클래스 매핑을 계획대로 적용



1번 변경의 근거 — 같은 연산의 지연 분포. 리눅스는 꼬리가 길다



2번 변경의 근거 — 우선순위를 뒤집으면 NPU 추론(25 ms)이 IMU 주기(20 ms)를 막는다

바뀌지 않은 것

이중 안전망이라는 구조와, '블랙아이스를 직접 맞히지 않고 위험도를 올린다'는 **1차 전략**, '노면 라벨과 무관하게 차량 거동으로 확정한다'는 **2차 전략**은 계획서 그대로다. 바뀐 것은 전부 그 구조를 더 확실히 증명하기 위한 수단이다.

데이터셋을 먼저 고르지 않았다. 각 헤드가 무엇을 정답으로 배워야 하는지를 정의한 뒤 거기에 맞는 공개 데이터셋을 매핑했다.

방어	필요한 정답(label)	데이터셋	활용
1차	건조 / 젖음 / 눈 / 결빙 클래스	RSCD	주 학습 (사전학습)
1차	실측 수막 두께 (센서 GT)	RoadSaW	반사도 헤드 근거
1차	눈 · 비 · 안개 + 조도	AI Hub	국내 도로 도메인 검증
2차	실측 6축 IMU + 노면 라벨	PVS	칼만 공분산 설정
2차	RGB-IMU 동기화 · 악조건	ROAD	비교 기준선
보조	실제 폭설 주행 씬	CADC · WADS	야간 · 악천후 점검
보조	자동 라벨 (차선 가시성)	CARLA	합성 데이터 생성

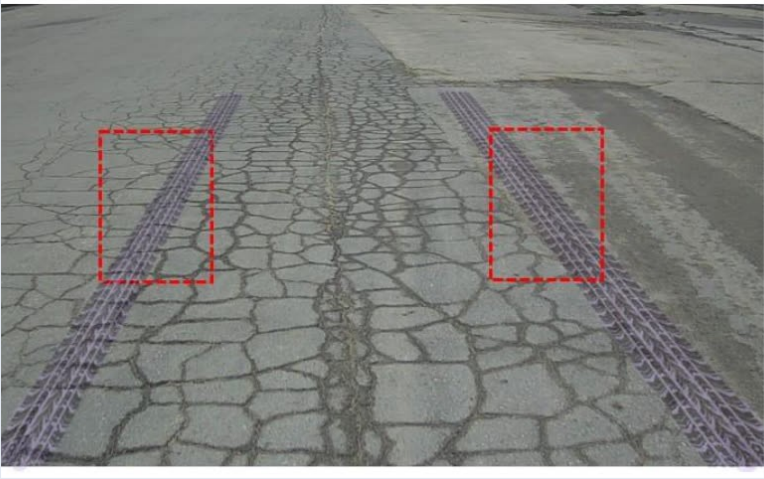
RSCD 27클래스 → 우리 4클래스

정상 ← 건조 (dry) **젖음** ← 젖음 · 물 고임 (wet · water)
결빙 위험 ← 결빙 · 녹은 눈 (ice · melted snow) **포트홀** ← 요철 '심함' 라벨

약 100만 장(공개 서브셋 37만 장) · 27클래스 = 마찰 6 × 재질 4 × 요철 3 · 베이징 약 700 km 실도로 주행(2022) · **240×360 패치**라 NPU 입력 크기에 맞는다 · 눈·얼음은 결빙 57,262 · 녹은 눈 64,263 · 신설 76,730장



RSCD 클래스별 샘플

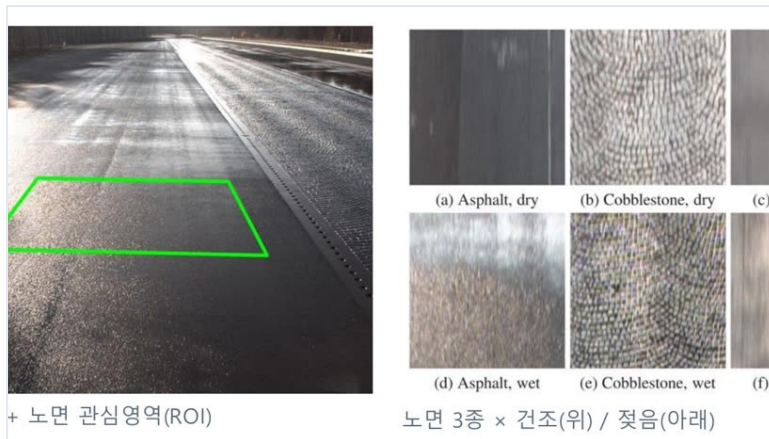


주행 영상에서 노면 영역만 잘라 패치로 쓴다

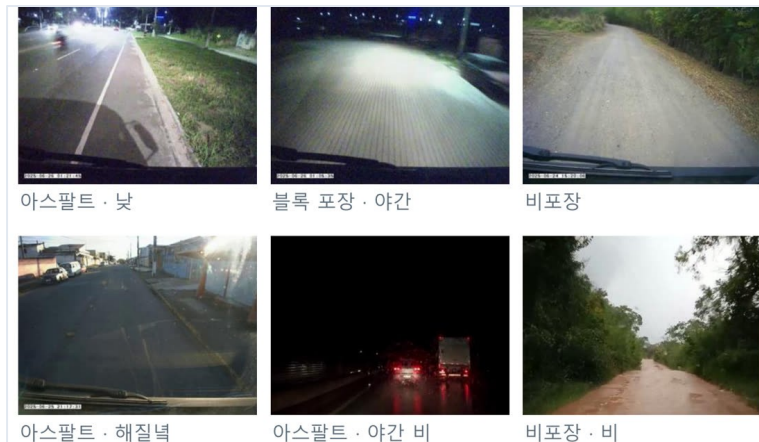


차량 전방 카메라 — 20~80 km/h 주행 촬영

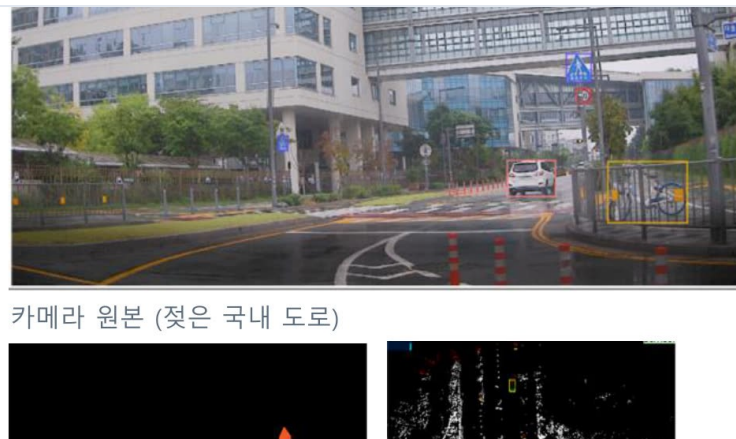
데이터셋 그림은 각 공개 데이터셋의 논문·공식 사이트에서 가져온 설명 목적의 인용이며, 라이선스는 저장소 NOTICE 에 명시했다.



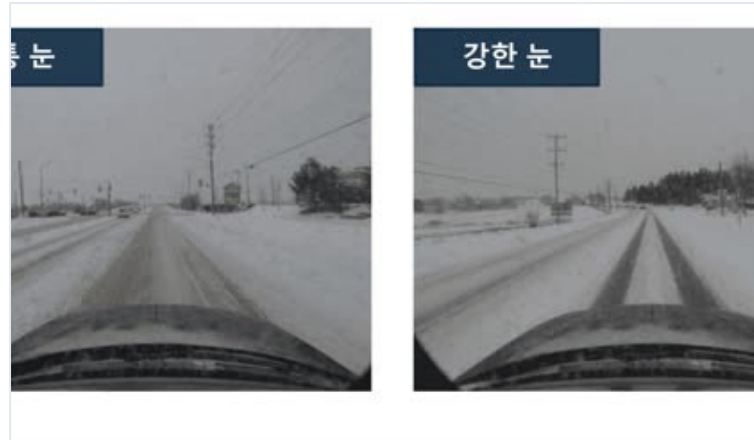
RoadSaW 12클래스 = 노면 3종 × 젖음 4단계. MARWIS 로 수막 두께 실측 → 반사도 회귀의 근거. 패치 약 72만 장



ROAD 카메라 30 fps + IMU 5개 400 Hz 동기, 약 115만 프레임. 야간·폭우·먼지 악조건



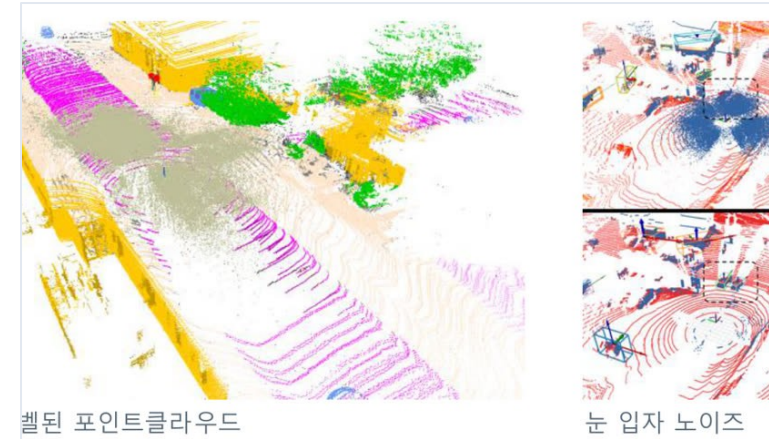
AI Hub 승용 자율주행차 악천후(71626). 카메라·라이다·레이더 + 2D 분할 라벨 + 3D 박스 — 우리 라벨 형식의 본보기



CADC 눈길 실주행 5.6만 장 · 라이다 7천 스윕 · 75개 장면 (캐나다 워털루)



PVS MPU-9250 IMU 100 Hz 를 대시보드·서스펜션 3곳에. 9세트 = 차량 3 × 운전자 3 × 경로 3



WADS 미시간 폭설 라이다. '내리는 눈 / 쌓인 눈'을 포인트별 라벨(36억 점)

△ RoadSaW 에는 눈·얼음이 없고, ROAD 의 클래스는 노면 종류(아스팔트/블록/비포장)라 결빙이 없다. 한계를 알고 역할을 나눠 썼다.

항목	값
백본	MobileNetV3-Small (ImageNet 사전학습)
입력	1×3×224×224 NCHW, int8
출력	4클래스 로짓 + 반사도 헤드 (576→64→1)
내보내기	ONNX opset 13, 고정 배치 1
양자화	int8 PTQ (QDQ), 가중치 채널별 · 활성화 대칭
입력 양자화	scale 0.018658448, zero-point -14
출력 양자화	scale 0.029451849
프레임당 전송	150,528 B
NPU	Neural-ART @ 1 GHz · 추론 25 ms
카메라 ROI	차량 전방 7.9 ~ 42.2 m (FOV 60°, 피치 -12°)

반사도 헤드를 따로 둔 이유

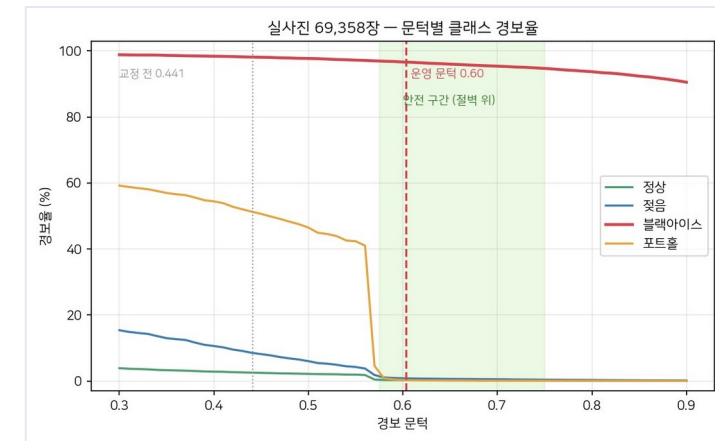
블랙아이스의 단서는 '무슨 노면인가'보다 '빛을 어떻게 되돌리는가'에 가깝다. 분류 로짓 하나로는 젖음과 결빙이 섞인다. RoadSaW의 실측 수막 두께를 정답으로 반사도를 회귀로 따로 배우게 하고, 융합 단계에서 분류 확률과 더한다.

양자화에서 겪은 것

표준 PTQ로는 stem 층의 활성화 분포가 넓어 int8에서 정확도가 떨어졌다. **stem 등화(equalization)**와 클리핑을 적용해 회복했고, QDQ 그래프를 u8→i8로 다시 쓰는 변환을 거쳐 Neural-ART가 받는 형태로 맞췄다.

시뮬레이션 화면이 아니다. **RSCD** 실사진을 **STM32N6** 에 그대로 넣고 추론·융합을 전부 보드에서 돌려 받은 판정이다.

실제 노면	장수	정답률	경보율 (문턱 0.603)
블랙아이스	6,340	96.1 %	96.6 %
마른 노면	19,018	80.7 %	0.3 %
젖은 노면	34,440	84.6 %	0.8 %
포트홀	9,560	90.2 %	0.3 %

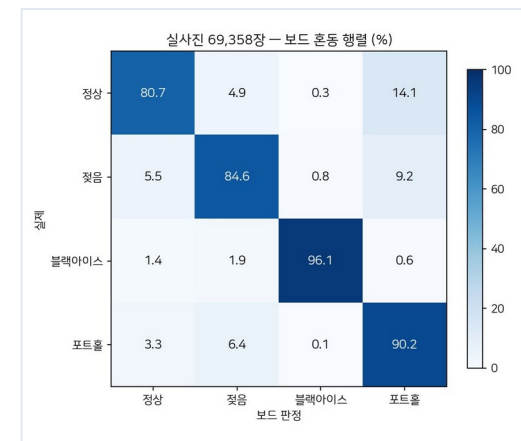


이 숫자가 왜 믿을 만한가

표본이 보드 바깥에서 만들어진 것이 아니다. 실사진을 **STM32N6** 에 그대로 넣어 추론·융합을 보드에서 돌리고, 돌아온 판정을 그대로 썼다.

장당 판정을 `logs/rscd_board_samples.jsonl` 에 남겨 두어, 문턱을 바꿔 다시 계산할 때 보드를 다시 돌릴 필요가 없다.

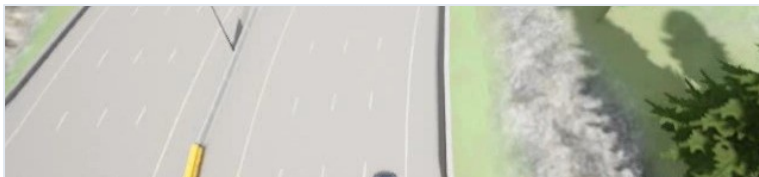
운영 문턱 결정 — 69,358장 전량 스윕



혼동 행렬 (행 = 실제, 열 = 보드 판정)

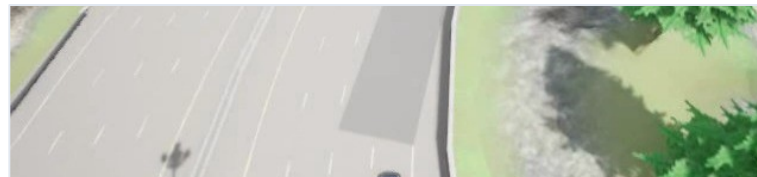
경보율은 보드의 alarm 플래그가 아니라 $risk \geq$ 문턱으로 계산했다. 보드 융합기의 히스테리시스(켜짐 0.441 / 꺼짐 0.341)는 연속 영상의 깜빡임을 막지만, 서로 무관한 낱장 사진을 이어 넣으면 앞 사진 상태가 넘어와 경보율이 부풀러지기 때문이다.

CARLA Town04 · 맑은 낮 · 40 km/h · 마찰 0.02 · 1차 방어 ON (보드 NPU 추론 + 융합)



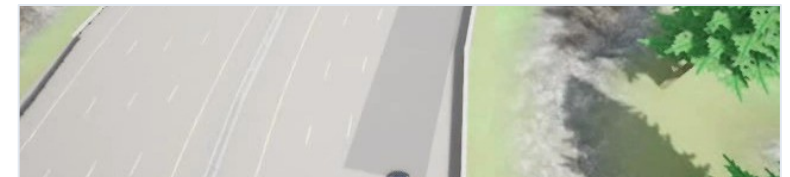
① 접근

40 km/h 주행. 위는 전방 카메라(모델 입력), 아래는 조감.
위험도 0.00 · DRIVE



② 1차 경보 6.10 s

위험도 0.443 → 문턱 돌파. 빙판 가장자리까지 11.4 m.
PRIMARY WARNING · BRAKE



③ 정지 7.14 s

빙판 23.6 m 앞에서 멈췄다.
빙판에 닿지 않았다 — 예방에 성공

같은 주행선에서 날씨 11종(맑음·흐림·젖은 노면·보슬비·폭우·해질녘·밤·비 오는 밤·눈 등)을 모두 돌렸다. 인식·정지 14회 · 경고 늦음 7회 · 미인식 10회 · 오경보 1회.

$$\text{risk} = \alpha \cdot \text{p_ice} + \beta \cdot \text{반사도} + \gamma \cdot (1 - \text{차선 가시성})$$

기상 · 위치 맥락	α (분류)	β (반사도)	γ (차선)
교량 · 새벽 등 결빙 위험 높음	0.35	0.45	0.20
일반	0.50	0.30	0.20
저위험	0.55	0.15	0.30

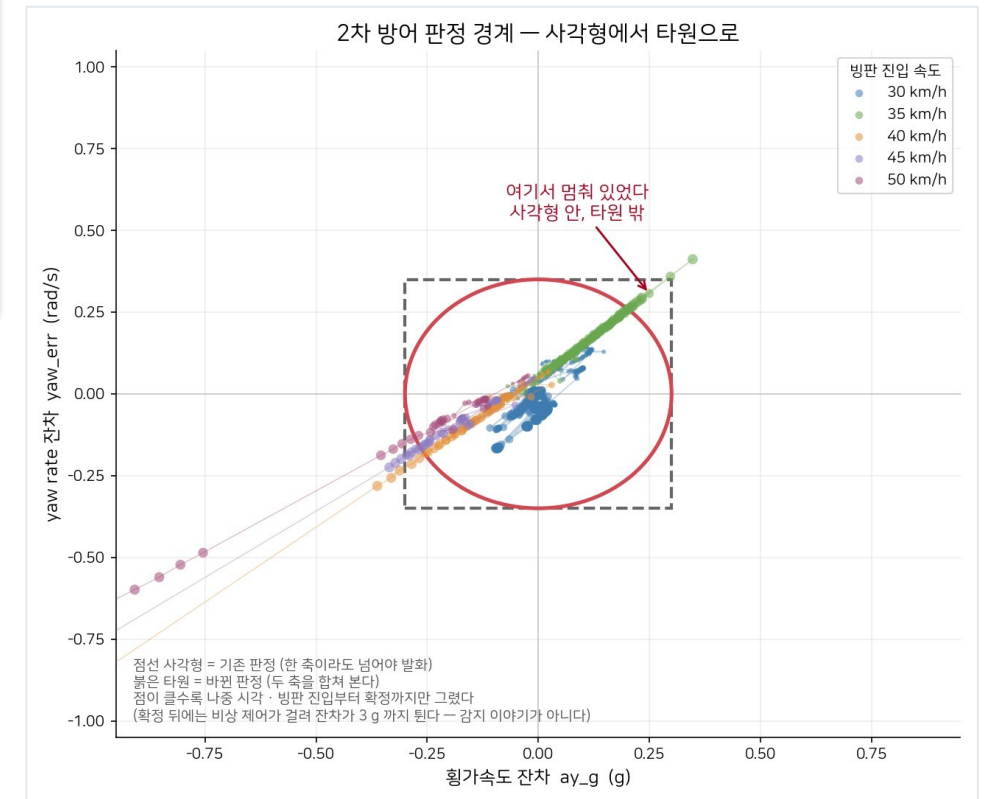
설계에서 고친 두 가지

- ① 재정규화. 쓸 수 없는 신호에 0을 넣으면 안 된다. 0은 중립값이 아니라 최솟값이라 위험도 상한이 잘린다. 그래서 쓸 수 없는 신호는 분모에서 빼고 남은 신호로 다시 정규화한다.
- ② 문턱 재교정. 초기 0.441 에서는 젖은 노면 오경보가 40 %였다. 실사진 69,358장 스윕으로 절벽 구간을 찾아 **0.603** 으로 올렸고 오경보가 0.8 %로 내려갔다. 폭우는 강수 게이트로 따로 분리했다.

판단 절차

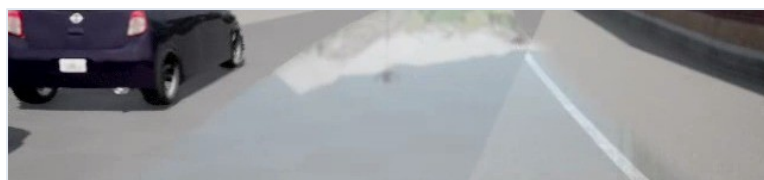
1. IMU 50 Hz 에서 횡가속 a_y · yaw rate · 종가속을 받는다
2. 2상태 칼만 필터(값 · 변화율)로 잡음을 거른다 — $Q = 2000/3000$, $R = 0.16/0.0005$
3. 자전거 모델로 '이 속도·이 조향이면 나와야 할 yaw rate'를 계산한다 (고속 언더스티어 보정 $1/(1+(v/v_{ch})^2)$, $v_{ch} = 17$ m/s)
4. 실측과 모델의 잔차가 타원 밖이면 미끄러짐 — 3샘플(60 ms) 연속이면 확정

항목	값	왜 이 값인가
축거 L	2.7 m	자전거 모델
조향 지연 τ	0.06 s	급조향 시 '모델이 즉답한다'는 가정이 오탐을 만들었다
횡가속 임계	0.30 g	
yaw 오차 임계	0.35 rad/s	
판정 규칙	타원	직사각형은 모서리에서 저속 지연이 생겼다
저마찰 트리거	제동 ≥ 0.3 이 0.3 s 지속 + 감속 < 1.2 m/s ²	필터 지연 구간의 오탐 방지



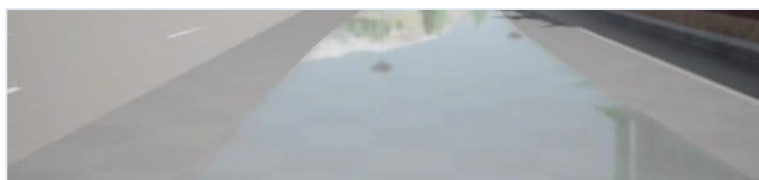
판정 경계 — 직사각형(점선) 대 타원(실선). 모서리에 걸리던 저속 구간이 타원에서 사라진다

CARLA Town04 · 맑은 낮 · 40 km/h · 마찰 0.08 · 1차 방어 OFF · 2차 판정 주체 = STM32N6 보드



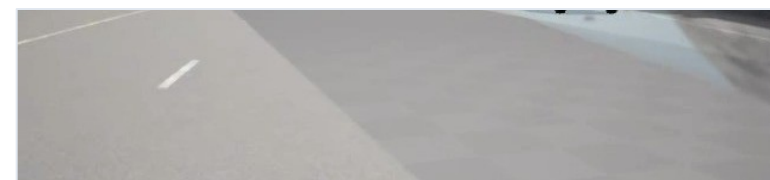
① 빙판 진입 5.84 s

1차를 끈 상태(미인식 가정). 38.7 km/h 로 그대로 들어간다.
위는 조감(차선 위치), 아래는 전방 카메라. 주변 차량 8대



② 미끄러짐 확정 7.62 s

STM32N6 RTOS 가 판정. $a_y = -0.364$ g, yaw 오차 -0.282
진입 1.78 s 만에 확정 → hard_stop



③ 차선 유지하며 정지 11.68 s

왼쪽 차로가 비어(gap 999 m) evade_left 로 전환 후 정지.
차선 안에서 멈췄다 — 가로 미끄러짐도 충돌도 없다

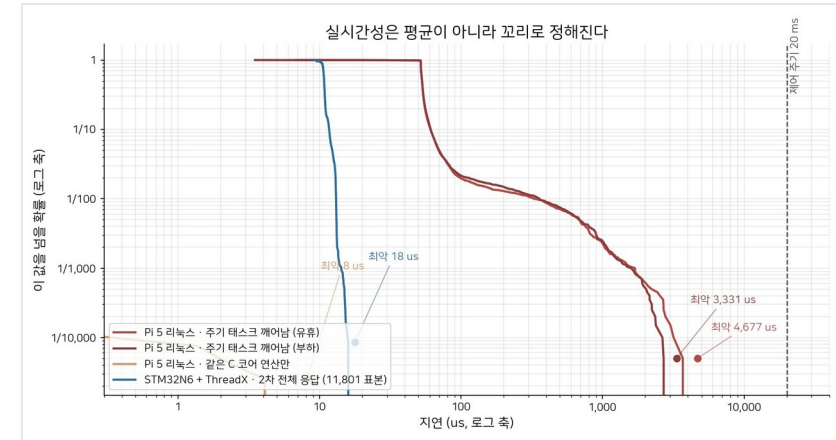
2차 방어 발동 62회(보드 판정 53 · 호스트 9). 진입→확정 평균 2.41 s, 확정→정지 평균 3.78 s. 비상 제어 모드 분포 — 차선유지 29 · 정지 37 · 최대제동 20 · 우회피 11 · 좌회피 11.

같은 연산을 Pi 5 리눅스가 평균 146배 빠르게 한다. 그런데도 RTOS 보드를 쓴다. 안전 기능의 기준은 평균이 아니라 최악이기 때문이다.

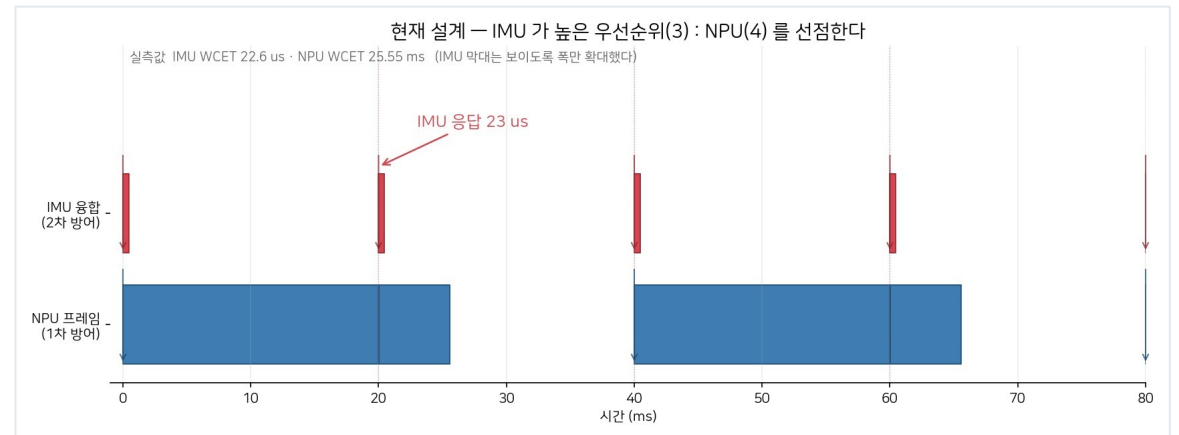
플랫폼	표본	중앙값	최악
Pi 5 + Linux · 유헤	20,000	69 μ s	5,790 μ s
Pi 5 + Linux · 부하	20,000	68 μ s	5,429 μ s
STM32N6 + ThreadX (응답 전체)	49,405	12.4 μ s	22.6 μ s

이것이 왜 치명적인가

제어 주기는 20 ms 다. 리눅스의 최악 지터 5,790 μ s 는 한 주기의 29 %를 한 번에 먹는다. 40 km/h 에서 5.8 ms 는 6.4 cm 지만, 미끄러짐이 시작된 뒤의 제어 루프에서는 그 한 번이 차선 유지와 제어 상실을 가른다.



지연 분포 CDF — 리눅스는 꼬리가 길다



RM 스케줄 — IMU(우선순위 3)가 NPU(4)를 선점한다

우선순위를 뒤집으면(NPU가 IMU보다 높으면) 25 ms 추론이 20 ms 주기를 막아 스케줄 자체가 불가능해진다 — 근거: docs/evidence/10_스케줄가능성_분석.md

1차·2차를 모두 끈 채 같은 빙판·같은 속도·같은 주행선으로 들어갔다.



차선 이탈 → 가로 미끄러짐 12.02 s 이탈(횡오프셋 -1.45 m, 방향 오차 -41°) → 14.82 s 에 -99°. 차체가 차로를 가로질러 90° 가까이 돌아 있다



방호벽 충돌 순간의 전방 카메라 11.42 s 이탈 → 12.22 s 에 -98° → 12.32 s 에 12.0 km/h 로 벽에 충돌. 화면이 방호벽으로 가득 찼다

'스핀'은 차선 대비 방향 오차가 86°를 넘은 순간으로 정의했다. 실측값은 -88.4°에서 -99.2° 사이로, 차체가 역방향을 본 것이 아니라 차로를 가로질러 돌아간 상태다. 주행별 이벤트는 docs/data/events/events_*_nodefense*.json 에 그대로 있다.

결과

9건 전부 차선을 벗어났고, 차체가 진행 방향과 88~99° 어긋난 채 가로로 미끄러졌다.

그중 5건은 12~17 km/h 로 방호벽에 부딪혔다.

주변 차량이 있던 4건은 충돌 전에 미끄러짐으로 끝났다.

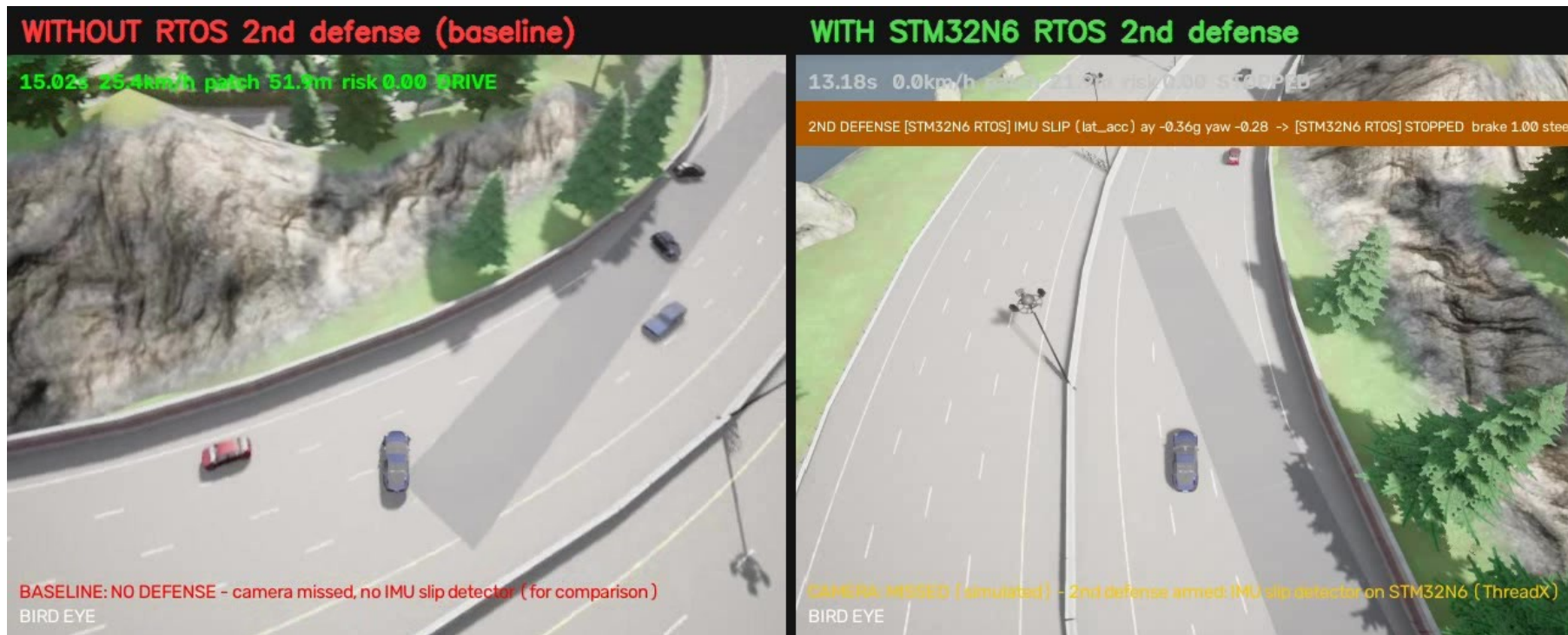
기준선의 가정

제어를 잃은 뒤에는 운전자 입력을 모형화하지 않는다. 비교 대상은 '우리 시스템이 개입하느냐'이지 '운전자가 얼마나 잘 대처하느냐'가 아니기 때문이다.

그래서 차선 이탈이 확정되면 자율주행을 떼고 관성에 맡기며, 정지하거나 충돌하거나 4초가 지나면 주행을 끝낸다.

실제 운전자는 제동을 시도하므로, 기준선의 결과는 '아무 보조도 없을 때의 물리적 귀결'로 읽어야 한다.

같은 날씨 · 같은 주행선 · 같은 빙판 · 같은 주변 차량 8대. 차이는 2차 방어 하나뿐이다.



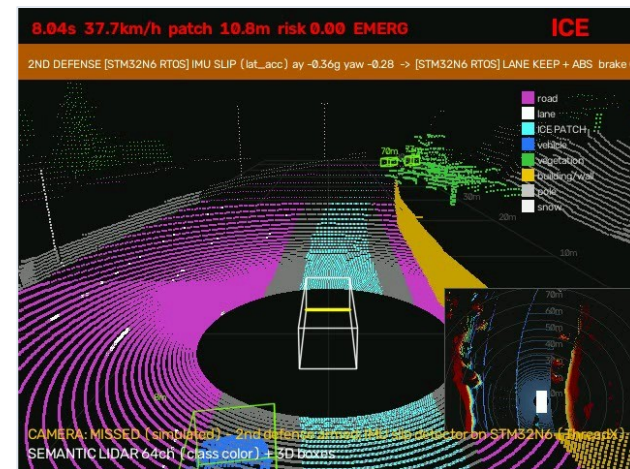
왼쪽 · 방어 없음

15.0 s · **25.4 km/h** · 12.0 s 에 차선을 이탈해 제어를 잃었고, 차체가 진행 방향과 어긋난 채 차로를 가로질러 미끄러지는 중이다. 뒤따르던 차량이 그대로 접근하고 있다.

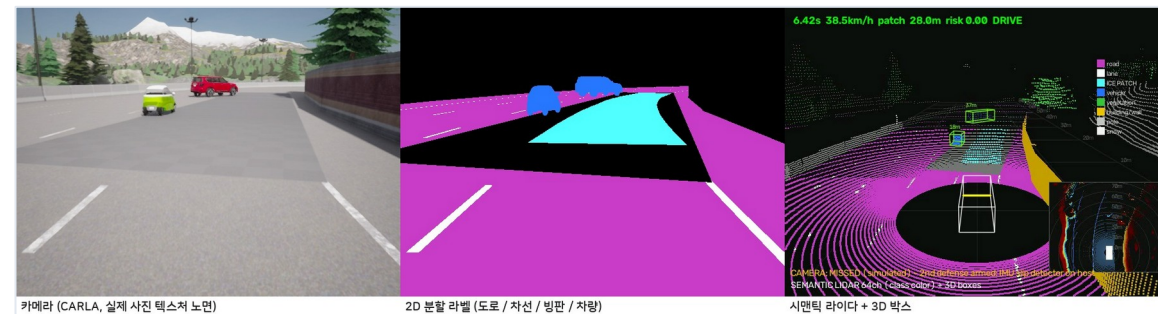
오른쪽 · STM32N6 RTOS 2차 방어

13.2 s · **0.0 km/h** · IMU SLIP (lat_acc) a_y -0.36 g, yaw -0.28 → STOPPED
차선 안에서, 빙판을 벗어난 지점에 정지했다.

날씨 4종(맑은 낮 · 밤 · 젖은 노면 · 눈)에서 같은 비교를 만들었다. 전체 영상은 저장소 media/ 와 logs/carla_demo/정리/G_비교_방어없음_vs_RTOS/ 에 있다.



시맨틱 라이다 64채널 — 청록이 빙판 구간, 초록 상자가 차량 3D 박스



날씨 11종에서 1차 경보가 난 같은 순간 — 맑음 · 흐림 · 젖은 노면 · 보슬비 · 폭우 · 해질녘 · 밤 · 비 오는 밤 · 눈. 같은 주행선, 같은 빙판에서 날씨만 바뀌어 돌렸다

카메라 · 2D 분할 라벨 · 라이다 — 도로/차선/빙판/차량 라벨을 자동 생성한다 (AI Hub 형식 참고)

주행 108회 전체의 이벤트 JSON 을 저장소에 함께 올렸다. CARLA 도 보드도 없이 scripts/analysis/summarize_runs.py 만으로 집계표를 재생성할 수 있다 — 손으로 적은 숫자가 아니라는 증명이다.



① 경보 거리는 기하 문제다

모델이 요구하는 최소 80 px 과 ROI 해상도 91 px 이 경보 거리를 정한다. 알고리즘이 아니라 해상도를 올리는 것이 먼저다.



② 2차는 예방이 아니다

60 km/h. 1차가 경고했지만 제동이 늦어 진입했고, 2차가 0.26 s 만에 개입했는데도 **39.2 km/h** 로 앞차와 충돌했다.



③ 오경보도 난다

밤. 빙판이 **41 m** 밖(ROI 7.9~42.2 m 바깥)에 있는데 어두운 노면을 얼음으로 보아 위험도 0.52 로 경보가 났다.



④ 질감 도메인 갭

RSCD 실사진 질감을 CARLA 노면에 입히면, 같은 모델이 그 얼음에 반응하지 않는다. 재수집·재학습이 다음 과제다.

그 밖에 남은 것

- **Hailo-10H** 교차 검증은 Dataflow Compiler 미확보로 열지 못했다. 1차는 STM32N6 Neural-ART 단독 결과다.
- **D435i** 실측 **IMU** 노이즈는 PVS 공개 데이터로 대체했다. 실물 연결 측정은 남은 과제다.
- 기준선은 제어 상실 후 운전자 입력을 모형화하지 않는다. '아무 보조도 없을 때의 물리적 귀결'로 읽어야 한다.
- 젖은 노면 경보율 0.8 %는 더 낮출 여지가 있다. β (반사도 가중치)와 문턱의 재조정이 필요하다.

무엇을 만들었나

블랙아이스 라벨이 없다는 데이터의 한계에서 출발해, 카메라 한 겹에 기대지 않는 이중 안전망을 STM32N6 한 보드 위에 올렸다. 1차는 빙판에 닿기 전에 세우고, 1차가 틀려도 2차가 최악 22.6 μ s 안에 반드시 받는다.

재현

모든 숫자는 실측에서 자동 생성된다.

```
python3 scripts/analysis/summarize_runs.py # 집계표 재생성
python3 -m pytest -q # 단위 테스트 19개
python3 fw/npu_lib/test_slip_core.py # C 코어 ↔ 파이썬 동치
```

향후 계획

1. 경보 거리 확대 — ROI 해상도를 올려 1차가 보는 거리를 늘린다. 2차 의존도를 낮추는 가장 직접적인 길이다
2. 실사진 질감 재학습 — RSCD 질감을 CARLA 노면에 입혀 재수집하고 도메인 갭을 닫는다
3. Hailo-10H 교차 검증 — Dataflow Compiler 확보 후 동일 모델을 두 NPU 에서 비교
4. D435i 실측 IMU — 실물 노이즈로 칼만 공분산을 재조정

소스 코드

[github.com/gkgk0119gmail-arch/
2026ESWContest_mobility_ARTS](https://github.com/gkgk0119gmail-arch/2026ESWContest_mobility_ARTS)

MIT 라이선스 · 영상·그림·이벤트 데이터 포함 · README 에서 바로 재생된다

근거 문서 19건 · 결과 그림 29장 · 주행 이벤트 108건을 저장소에 함께 올렸다.

